Pitho

How I build Pitho

The process, not the product. Steps, tools, and the rules learned by breaking things.

Hermione Gogou

The loop

1. **Read the docs of the thing you are using, first.** Frameworks move faster than memory. Before writing against a framework, a payment provider, or a platform API, open its current documentation and check the shape of the thing you are about to call. Half of the worst bugs here came from assuming an API worked the way a similar one did.
2. **Build the smallest honest version.** Not a mock, not a stub with a TODO. The smallest thing that really works end to end, including the failure path.
3. **Put it in front of real conditions early.** Real screen sizes, real files, real payments, real email. A feature that only works on a fast laptop with a fresh account is not finished.
4. **Reproduce before fixing.** Never patch a bug you have not seen happen. Drive the app until it fails in front of you, and measure what actually changed. Guessing produces fixes that do not fix anything.
5. **Verify in production after every deploy.** Send real requests. Check that the gates answer the way they should when things go wrong, not just when they go right.
6. **Write down what surprised you.** In the code, next to the surprising part, in plain language. The comment explains why, never what.

Tools

One tool per job, chosen so it can be swapped later.

Job	What we use	Why it was picked
App framework	Next.js (App Router)	Server and client in one place, route handlers for APIs
Hosting and runtime	Cloudflare Workers	Fast everywhere, no servers to keep alive
Deploys	Push to `main`	The build runs on the platform. No deploy script to forget
Database	Postgres (Neon), over HTTP	Serverless friendly, no connection pool to babysit
File storage	Object storage bound to the app	No API keys to issue or rotate
Transactional email	Resend	Simple API, domain verification, honest error messages
Inbound email	Cloudflare Email Routing	Real addresses on the domain, forwarding to a person
Payments	Polar (merchant of record)	They are the legal seller and handle tax
UI components	shadcn-style components on Tailwind	Owned in the repo, editable, no black boxes
Icons	One icon set, one wrapper component	Swapping sets is a one-file change

Rules that keep it standing

One seam per external system. Everything that talks to the outside world goes through a single small module: storage, email, payments, uploads. The rest of the app never imports the vendor directly. Swapping a provider then means editing one file, not hunting through fifty.

Secrets live in the platform's secret store. Nothing else. Not in version-controlled config, not in a variables block, never pasted into chat. A plain config variable gets overwritten by the next deploy, and a leaked one cannot be un-leaked. If a secret is ever exposed, rotate it rather than hoping.

Identifiers are not secrets. Product IDs, account IDs, and email addresses belong in config where a deploy cannot wipe them and anyone can read them.

Webhooks are idempotent and fail closed. Providers retry, so the same event arrives more than once and handling it twice must be harmless. Verify the signature before parsing the body. Verification that throws must return "no", not a server error, or the provider retries into the void and real events are lost silently.

Every critical path needs a floor. If the clever route is unavailable, the user still gets through by a slower route that always works. A path with no fallback is an outage waiting for a bad afternoon.

Fail loudly at the edges, quietly in the middle. A missing configuration should say exactly what is missing. A cosmetic failure should never take down the thing the user came to do.

Changing one screen size must prove the other is unchanged. Use breakpoints rather than branching in code, then check both sizes and confirm the untouched one really is untouched.

Copy is part of the build. Short, plain, and it says what actually happened. No apology, no jargon, no blame. Error text is read at the worst moment, so it should name the next action.

Verifying

- **Measure, do not assert.** When something looks broken, check the real state: the server's copy of the record, the computed style, the provider's own dashboard. Several "bugs" here dissolved on inspection, and one that looked cosmetic turned out to be deleting data.
- **Check the failure paths on purpose.** Unsigned webhook, expired link, wrong plan, no session. These are the paths that decide whether a bad day stays small.
- **Adversarial review before anything touching money or auth ships.** Have the work read by someone whose job is to break it, with instructions to find the case where it is wrong. Fix what survives that.
- **Clean up after testing.** Delete the test records, revoke the test keys, refund the test payment.

Working with an agent

Most of this was built by an agent working in the repo. What makes that go well:

- **Put the house rules in a file the agent always reads.** One short file at the repo root, saying what is different about this project and what to check before writing code. It is the difference between an agent

that reads your framework's docs and one that writes last year's API from memory.

- **Ask for evidence, not confidence.** "It works" means nothing. "Here is the request I sent and the status it returned" means something. An agent that cannot show you the check has not done it.
- **Let it reproduce the bug before it fixes the bug.** Same rule as for people, and easier to enforce, because you can ask to see the failing measurement.
- **Never hand it credentials.** Not in chat, not in a file it can read. It can do everything around a secret: name it, wire it, tell you which command to run, verify afterwards that the system accepted it.
- **Expect it to be wrong sometimes, and make that cheap.** Small commits with honest messages, so a bad change is one revert rather than an archaeology project.

Data that changes shape

The shape of stored records changes as the product grows, and there is never a good moment to stop and migrate everything.

- **Migrate lazily, on read.** Stamp records with a version. When one is read, bring it up to date in memory and save the new shape the next time it is written. No downtime, no migration script, no big-bang.
- **Remember what was deleted.** If a migration adds missing pieces, it will helpfully re-add the thing the user deliberately removed. Deletions have to be recorded, or they undo themselves.
- **Keep the storage interface tiny.** Ours is get, put, delete, list, plus the same for files. A small interface is one you can reimplement on a different database in an afternoon.

Limits, quotas, and honest promises

- **Never advertise a limit the platform cannot deliver.** Check the real ceiling of your host before publishing a number. We shipped a limit once that the runtime refused to accept, which turns every attempt into a support ticket.
- **Sum, do not count.** Work out usage by adding up what is actually stored, not by keeping a counter. Counters drift the first time something fails halfway.
- **Refuse before you store, and say what is left.** "That file is 40MB and you have 12MB free" is a useful refusal. "Upload failed" is not.
- **Stream large files straight through.** Reading a whole upload into memory costs twice its size and fails at the worst time. Pass the stream to storage.
- **Know what a customer costs you.** Storage and bandwidth per account, times the number you hope to have. It decides the price, and it is easier to decide before anyone is paying.

Sessions on real devices

- **Links opened in an app open in that app's browser.** A sign-in link tapped inside a mail client creates the session in that client's own browser, with its own cookies. The user then opens the normal browser and appears logged out. Nothing is broken, and it will still read as broken to them.
- **Assume the session is missing.** Any page reachable from an email or a shared link should work, or explain itself, without one.

Undo and destructive actions

- **Take the undo snapshot after the change lands, not while the click is handled.** Otherwise "undo" reverts past the thing you just did. Ours deleted a whole section, and saved that deletion, on a single Escape.
- **Destructive actions get confirmed, and never sit on the primary target.** On touch there is no hover to hide them behind, so they belong in a menu.
- **Autosave and undo have to agree.** If a revert writes to the server, then a wrong revert is data loss rather than an inconvenience.

Know what you are not doing

Write down the gaps you are choosing to live with, and why, in the repo. Rate limits that only hold per instance, a check that is enforced in the UI but not the API, a plan whose limits are not yet different. Written down, they are decisions you can revisit. Undocumented, they get rediscovered every few weeks and argued about again.

What stays human

Some steps do not get automated, not because they are hard but because the cost of being wrong is not recoverable:

- Creating credentials, and putting them where they belong
- Moving money: charges, refunds, payouts, cancellations
- Creating accounts and accepting terms
- Publishing anything public under someone's name
- Deleting data that is not obviously disposable

Everything else is fair game to automate, script, or hand to an agent.